Formula Uno

The Codebase, File by File

A guided tour of every file in the repository: what it is, what its key functions do, and how it connects to the rest of the system. Files are grouped by role and tagged **CURRENT** (in active use), **NEW** (the Stage-1 API), or **LEGACY** (early scaffolding that predates the pipeline and no longer runs against today's code).

The repository at a glance

```
Formula Uno/
|- run_bahrain.py          [CURRENT] entry: Bahrain full pipeline
|- run_spa.py              [CURRENT] entry: Spa full pipeline
|- run_multi_circuit.py    [CURRENT] entry: combined multi-circuit model
|- run_api.py              [NEW]     entry: launch the FastAPI server (Stage 1)
|- run_features.py         [LEGACY]  early feature-inspection script
|- run_model.py            [LEGACY]  early train-only script
|- run_engine.py           [LEGACY]  early end-to-end script
|- run_all.py              [LEGACY]  early 2023-&gt;2024 single-year prototype
|- data/                             FastF1 cache + data/models/*.pkl
|- src/
   |- ingestion/load_session.py      load race/qualifying from FastF1
   |- features/build_features.py     raw laps -&gt; clean feature table
   |- model/train.py                 XGBoost train / save / load / importance
   |- engine/race_engine.py          lap-by-lap race simulation
   |- analytics/compare.py           race: simulated vs actual, metrics
   |- analytics/qualifying.py        qualifying: extract, predict, report
   |- pipeline.py                    orchestrates everything
   |- api/predict_service.py [NEW]   compute layer for the API
   |- api/app.py             [NEW]   web layer (FastAPI endpoints)
   |- __init__.py (x7)               empty package markers
```

The sections that follow walk each file in the order data actually flows through the system: ingestion, features, model, engine, analytics, the orchestrator, then the new API layer, and finally the entry-point scripts.

src/ — the core package

Everything under **src/** is the reusable library. The seven **__init__.py** files are empty; their only job is to mark each folder as an importable Python package so that *from src.model.train import train* works.

src/ingestion/load_session.py — CURRENT

The only module that talks to FastF1. Loads a session and hands back its laps and results as pandas DataFrames; enables the on-disk cache so each session is downloaded only once.

`fastf1.Cache.enable_cache`	Run at import — points FastF1's cache at the data/ folder.
`_load(year,event,type)`	Fetches a session with telemetry/weather/messages off; returns {session, laps, results}.
`load_race / load_qualifying`	Thin wrappers for the 'R' and 'Q' session types. event may be a round number OR a name.
`inspect_laps(laps)`	Debug helper: prints shape, columns, a sample, lap-time stats, drivers and compounds.

Connects: downstream of FastF1; its output feeds build_features, the race engine, and the API.

src/features/build_features.py — CURRENT

The heart of feature engineering: turns messy human-readable laps into a clean 10-column numeric table the model can learn from. Also builds the categorical encoders.

`constants`	COMPOUND_ORDER, REG_ERA_START=2022, START_FUEL_KG=105, QUAL_FUEL_KG=5, FEATURE_COLS (10), TARGET_COL.
`build_encoders(sessions)`	Fits Driver/Team/Track LabelEncoders on the UNION of all sessions (prevents unseen-label crashes).
`_drop_bad_laps`	Keeps only trustworthy laps: IsAccurate, not deleted, present fields, and drops Safety-Car/VSC laps.
`_encode_compound / _encode_categoricals / _encode_track`	Map compound to an ordinal and driver/team/track to integers using the pre-fit encoders.
`_merge_grid_position`	Joins each driver's starting grid slot onto their laps.
`build_feature_frame(... year)`	Race pipeline: applies all steps and derives YearEncoded, FuelLoad, TyreLifeSquared.
`build_qualifying_feature_frame(...)`	Qualifying variant: fuel fixed at ~5 kg, no Position filter, grid from qualifying result.

Connects: consumes load_session output + encoders; its tables feed train.py.

src/model/train.py — CURRENT

Training, persistence and inspection of the XGBoost models. Same code trains both the race and qualifying models — only the data differs.

`train(df,feature_cols,target)`	Builds the XGBRegressor (2000 trees, lr 0.02, depth 6, min_child_weight 5, 0.8 subsample/colsample), runs 5-fold CV, fits, reports train MAE/RMSE.
`save_model / load_model`	Pickle a model to / from data/models/{name}.pkl.
`feature_importance(model)`	Prints each feature's gain-based importance, ranked.

Connects: consumes feature frames; produces the .pkl models used by the engine and the API.

src/engine/race_engine.py CURRENT

The deterministic simulator: turns the lap-time model into a full Grand Prix, lap by lap, letting finishing order emerge from accumulated time.

`DriverState (dataclass)`	Per-car mutable state: identity, compound, tyre age, cumulative time, position, history, retired flag.
`RaceEngine.__init__`	Holds the model, encoders, feature columns, track code and the year encoding.
`_build_driver_states`	Initialises each car's starting compound, tyre age and grid position from the real race.
`_build_pit_schedule`	Reads real pit stops; stores actual in-lap/out-lap times and the new compound.
`_predict_lap(state,lap,total)`	Assembles the 10 features (incl. fuel and tyre² recomputed live) and calls model.predict.
`simulate(raw_laps,results)`	Main loop: three lap cases (out-lap/in-lap/normal), updates state, sorts by cumulative time to assign positions; returns a per-lap history DataFrame.

Connects: uses a model + encoders; consumes raw laps; its output feeds compare.py and the API.

src/analytics/compare.py CURRENT

Scores a race simulation against reality using two independent metrics: finishing-position error and per-lap time error.

`final_classification(sim)`	Reads the finishing order off the simulation's last lap.
`compare_positions(sim,results)`	Simulated vs actual position per driver, plus the absolute error.
`lap_time_rmse(sim,raw_laps)`	Per-driver RMSE between simulated and real lap times (clean laps only).
`gap_evolution(sim,a,b)`	Time gap between two drivers across every lap.
`print_summary(...)`	Console report combining the above.

Connects: consumes simulate() output + results; used by pipeline.py and reused by the API compute layer.

src/analytics/qualifying.py CURRENT

Validates qualifying separately from the race: extract each driver's best lap, predict it, and rank to produce a predicted grid.

`extract_best_qualifying_laps(q)`	Each driver's single fastest clean qualifying lap — the one that set their grid slot.
`predict_qualifying(...model...)`	Model-agnostic: predicts each best lap (fuel fixed 5 kg), ranks by predicted time into a grid. Pipeline passes the dedicated qualifying model.
`print_qualifying_report(df)`	Console report: grid accuracy and lap-time RMSE.

Connects: consumes a qualifying session + a model; used by pipeline.py.

src/pipeline.py — CURRENT

The orchestrator — the spine that wires every layer together for a run. This is what the entry scripts actually call.

`run_circuit(event,label,train_years,test_year,...)`	One circuit end-to-end: load all sessions, build encoders, build race + qualifying feature tables, train BOTH models and save them, simulate every training year in-sample, simulate the test year out-of-sample, and run the qualifying prediction.
`run_multi_circuit(circuits,train_years,test_year)`	Trains ONE race model and ONE qualifying model across several circuits combined (shared encoders), then reports per circuit.

Connects: called by run_bahrain / run_spa / run_multi_circuit; calls every other src module.

src/api/ — the Stage-1 serving layer (new)

These two modules turn the library into a callable web service. The design rule is a hard split between **compute** (no HTTP) and **transport** (only HTTP).

src/api/predict_service.py — NEW

The compute layer. Loads artifacts, runs a simulation, and returns a plain dict — deliberately knowing nothing about HTTP, so the frontend, RAG assistant and agent can all reuse it later.

`TRAIN_YEARS / SERVE_YEARS`	2022-24 trained; 2022-25 servable.
`CIRCUITS`	Registry of circuits that have a saved model on disk (currently just Bahrain).
`_get_artifacts(circuit)`	Rebuilds the encoders deterministically (matching the trained model) and loads the .pkl; caches both in memory.
`predict_race(circuit,year)`	Simulates the race, computes metrics via compare.py, returns a structured dict. Result cached per circuit+year.
`available_circuits()`	Lists what the API can serve.

Connects: reuses race_engine + compare + train.load_model; called by app.py.

src/api/app.py — NEW

The web layer (FastAPI). Defines endpoints, validates I/O with Pydantic, and maps errors to HTTP status codes. Contains no racing logic at all.

`Pydantic schemas`	DriverResult, Metrics, PredictionResponse — validate the response and auto-generate the /docs UI.
`GET /health`	Liveness check plus which circuits and years are available.
`GET /predict/race`	Calls predict_race(); returns the predicted order. KeyError->404, ValueError->422.

Connects: imports predict_service; served by run_api.py.

Entry-point scripts (repository root)

Thin scripts you actually run. The current four address races by NAME (never round number).

run_bahrain.py — CURRENT

Runs the full pipeline for Bahrain: train 2022-24, test 2025.

`run_circuit(...)`	event='Bahrain Grand Prix', train_years=[2022,2023,2024], test_year=2025.

Connects: calls pipeline.run_circuit.

run_spa.py — CURRENT

Same as Bahrain but for Spa (event='Belgian Grand Prix'). Note: its model is not currently persisted on disk, so the API cannot serve Spa until this is run once.

`run_circuit(...)`	event='Belgian Grand Prix', train_years=[2022,2023,2024], test_year=2025.

Connects: calls pipeline.run_circuit.

run_multi_circuit.py CURRENT

Trains one combined model across Bahrain + Spa and reports each.

`run_multi_circuit(...)`	circuits=[Bahrain, Spa], train_years=[2022,2023,2024], test_year=2025.

Connects: calls pipeline.run_multi_circuit.

run_api.py NEW

Launches the FastAPI server with uvicorn on 127.0.0.1:8000. Open /docs for the interactive UI.

`uvicorn.run('src.api.app:app', ...)`	Starts the web server.

Connects: serves src/api/app.py.

HEADS UP — The four LEGACY runners — safe to archive

run_features.py, run_model.py, run_engine.py, run_all.py are early scaffolding from before pipeline.py existed. They call an *older* build_feature_frame signature (a tuple return, plus a summarise() helper) that no longer exists in the current code, so **they will not run today**. They are harmless but confusing — useful only as a historical record of how the project was first assembled step by step. Recommendation: move them into an archive/ folder or delete them, keeping only run_bahrain, run_spa, run_multi_circuit and run_api as the live entry points.

data/ — cache and artifacts

Path	What it holds
data/ (FastF1 cache)	Downloaded session data; first load fetches over the network, later loads read instantly.
data/models/laptime_bahrain_2022_2023_2024.pkl	The trained Bahrain race-pace model.
data/models/qualtime_bahrain_2022_2023_2024.pkl	The trained Bahrain qualifying model.
data/api_server.log	A stray log from API testing — harmless, deletable.

NOTE — Only Bahrain artifacts exist right now

Spa's models were removed during an earlier cleanup and only Bahrain was retrained, so no laptime_spa_*.pkl is on disk. Running run_spa.py once regenerates and saves them, after which Spa can be added to the API's CIRCUITS registry.

How the files connect — one run, end to end

```
run_bahrain.py
  -&gt; pipeline.run_circuit
       -&gt; load_session.load_race / load_qualifying      (raw laps + results)
       -&gt; build_features.build_encoders                 (Driver/Team/Track codes)
       -&gt; build_features.build_feature_frame            (race feature table)
       -&gt; build_features.build_qualifying_feature_frame (qualifying feature table)
       -&gt; model.train + model.save_model                (two .pkl models)
       -&gt; engine.RaceEngine.simulate                    (per-lap race history)
       -&gt; analytics.compare.print_summary               (position + lap metrics)
       -&gt; analytics.qualifying.predict_qualifying       (predicted grid)

run_api.py
  -&gt; api.app  (FastAPI)
       -&gt; api.predict_service.predict_race
            -&gt; load_session + build_encoders + load_model   (artifacts)
            -&gt; engine.RaceEngine.simulate
            -&gt; analytics.compare (metrics)  -&gt; JSON
```

Every current file has one clear job and one clear place in the flow — which is exactly why new layers (frontend, RAG, agent) can be added without disturbing the core.